Verifier-Assisted versus LLM-Only Pre-deployment Network-Change Validation: A Preregistered Paired Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We measured whether integrating a deterministic pre-deployment verifier with a bounded automated repair budget increases the fraction of intent-driven network-change proposals that pass semantic validation compared with accepting a single LLM candidate. In a preregistered paired design (study id cnsmp2026-llm-verifier-predeployment-001) we executed 300 paired intents (100 per difficulty stratum: low/medium/high) using gpt-5-mini and deterministic_netops_validator_v5. Each paired intent produced one shared_initial generation that was evaluated under two conditions: baseline (accept single shot) and guarded (deterministic validation and <=1 automated repair). Outcomes: baseline 299/300 unflagged passes (0.9966667), guarded 300/300 (1.0); contingency n11=299, n10=1, n01=0, n00=0; absolute paired difference = 0.0033333; exact McNemar two-sided p = 1.0; 95% paired bootstrap percentile CI = [0.00,0.01] (bootstrap_seed = 1729, B = 10000). Under the supplied synthetic suite and repair budget the single-shot generator was near ceiling, limiting observable deterministic rescue. We archive preregistration, execution, and analysis artifacts and interpret results with respect to estimand conditioning, validator coverage, repair budget, and operational deployment policies.**

I. Introduction

Generative large language models (LLMs) are being applied to network operations (NetOps) tasks such as intent→configuration translation, configuration synthesis, and automated repair. For pre-deployment network changes, semantic correctness properties—sequencing constraints, reachability, and policy preservation—are operationally critical and cannot be assured by superficial syntactic checks alone. A common engineering pattern is to pair generative models with deterministic validators and bounded automated repair (generate→validate→repair) to reduce operational risk. However, the measurable benefit of such guarded pipelines depends on the experimental estimand (what is held fixed), the generator single-shot quality, the validator's expressiveness, and the permitted repair budget.

We preregistered a paired experiment (study id cnsmp2026-llm-verifier-predeployment-001) to quantify the deterministic rescue achievable when the same single LLM candidate is evaluated under two treatments. For each intent the pipeline produced exactly one shared_initial candidate using the declared generator (gpt-5-mini) and compared: (a) baseline — accept the single candidate without repair, and (b) guarded — run a deterministic validator (deterministic_netops_validator_v5) and allow at most one automated repair which itself is re-validated. The primary research question: does adding a deterministic verifier plus at most one automated repair increase the probability that a single LLM candidate passes semantic pre-deployment validation compared with accepting that same candidate without repair?

Contributions of this artifact-grounded study: (1) a preregistered paired evaluation isolating deterministic rescue under a shared_initial estimand that conditions on a single LLM draw per intent; (2) a complete execution and analysis bundle including validator traces and the single automated repair transcript archived with the run; and (3) an evidence-grounded interpretation of estimator conditioning, ceiling effects, and operational implications for NetOps pipelines.

II. Related Work

Recent surveys and systems work document active efforts to apply LLMs to NetOps tasks including intent translation, configuration synthesis, automated repair, and tool-augmented orchestration [https://openalex.org/W4402742293; https://openalex.org/W4411015066; https://openalex.org/W4416927413]. Architectures that interpose validators or tools (planner-centric frameworks, ReAct variants, and generate→validate→repair pipelines) are proposed to reduce hallucination and semantic errors; prior empirical gains vary substantially with validator coverage and generation strategy [doi:10.1609/aaai.v40i40.40676; https://openalex.org/W4404240614].

Two methodological gaps motivated the present design. First, many prior comparisons conflate deterministic validator rescue with benefits obtained by drawing multiple independent model candidates: permitting resampling mixes generator variance effects with deterministic validation effects. Second, community benchmarks emphasizing semantic correctness (reachability and policy compliance) rather than only syntactic validity are limited [https://openalex.org/W4415071524; https://openalex.org/W4392875514], making operationally meaningful comparison difficult. Our paired shared_initial design isolates deterministic rescue from resampling effects, providing an operational bound for policies that constrain model calls per change request.

III. Methodology

Preregistration and primary estimand. The confirmatory design, analysis plan, inclusion/exclusion rules, and estimand were frozen in a machine-readable preregistration (study

id cnsmp2026-llm-verifier-predeployment-001; preregistration SHA-256 = 425cb1652354120cf48f686b964723d64c76d0a8abe53cd5b5ad0eaa1af71fc7). The primary estimand is the absolute paired difference in the proportion of unflagged verifier passes (guarded minus baseline) computed on N = 300 paired intents after applying preregistered exclusion rules. The analysis executor paired_binary_analysis_v1 (frozen) performs contingency counting (n11,n10,n01,n00), an exact two-sided McNemar test, and a paired percentile bootstrap to produce a 95% CI. Bootstrap parameters used in the confirmatory analysis were seed = 1729 and B = 10000 as recorded in the archived analysis artifacts.

Task generation, stratification, and semantics. The task bank was produced deterministically by deterministic_netops_task_generator_v5 into a synthetic_topology_suite stratified into equal-sized low/medium/high difficulty bins. Each task manifest entry encodes: (a) an explicit natural-language intent; (b) initial and expected_final state maps; (c) the required_sequence of field/interface edits that represent sequencing and safety constraints (e.g., must_be_down_for_fields); (d) a compact difficulty fingerprint (changed_interface_count, dependency_rule_count, required_command_count, transient_rule_count); and (e) preserve_unspecified_state and protected_interfaces invariants. The stratification was operationalized by generator metadata so that each stratum contained 100 paired cases in the main run. The synthetic suite implements deterministic reachability and topology invariants that the validator checks; this permits semantic scoring (not just syntactic acceptance).

Shared_initial protocol and generation model control. To isolate deterministic rescue from resampling benefits we implemented a shared_initial estimand: for each paired intent the pipeline produced exactly one shared_initial LLM candidate and reused that identical candidate for both conditions. The declared generator for shared_initial production was gpt-5-mini (execution/model_configuration.json records requested_model = "gpt-5-mini"). The execution contract limited initial_generation_calls_per_task = 1 and maximum_repair_calls_per_task = 1. These settings fix the model-call budget and ensure that any difference between conditions arises only from the deterministic validator/repair stage applied to the same candidate rather than from drawing alternative candidates.

Validator, scoring rule, and constrained repair operator. Semantic validation used deterministic_netops_validator_v5 under the preregistered full_intent_constraint_validation rule. That rule performs deterministic checks for: sequencing constraints (must_be_down_for_fields), required_command_count equality, preservation of unspecified state, forbidden-template protections (protected_interfaces), and a pre/post reachability invariant test encoded by the synthetic topology suite. Scoring was binary: an unflagged result (valid == true) denotes a pass; any violation or missing required command yields an automated_flag and counts as a failure for that pipeline call.

The guarded condition permitted at most one automated repair (repair budget <=1). The repair operator implemented a constrained edit policy restricted to localized, intent-conservative fixes: insertion of missing sequencing commands, reordering of adjacent commands to satisfy explicit required_sequence constraints, or minor syntactic normalizations that do not alter fields outside the allowed_touched_fields. Repairs were deterministically proposed and applied, then re-validated by the same validator; if the repaired candidate remained flagged it counted as a failure. The repair provider trace (archived) documents the exact edit and subsequent validator outcome for each repair invocation (the run used one repair call). The repair operator intentionally excluded large-scope rewrites or generator re-invocations to preserve the shared_initial estimand.

Contamination controls, negative controls, and stopping rules. The preregistration mandated one negative-control (intentionally malformed) task per difficulty stratum to exercise the contamination detectors and invariant checks. Contamination detection included deterministic invariant comparisons (pre/post reachability matrices) and negative-control pass/fail checks; items that passed a negative control would be flagged as contamination (none did). Exclusion rules: a pair is excluded from confirmatory analysis only if both pipeline calls for that pair fail due to provider/infrastructure outage; single-call failures are handled per the executor's complete_pair_primary rule (none occurred). The stopping rule would halt the run if >20% of attempted pairs were excluded due to dual-call outages; this did not trigger in execution.

Execution instrumentation, audit, and deterministic reconciliation. The run used adapter hosted_netops_gvr_v1 in execution_mode = scientific_confirmatory. Execution manifests record planned_episode_count = 600, completed_episode_count = 600, complete_pair_count = 300, model_calls_used = 301, and maximum_model_calls = 600. Provider auditing values archived with the run include hosted_model_call_event_count = 301, cache_miss_count = 301, and stage_counts showing shared_initial_generation = 300 and repair = 1. Deterministic reconciliation procedures verified pair accounting and call auditing and report all_deterministic_consistency_checks_passed = true. The execution and reconciliation artifacts (execution/raw_results.jsonl, execution/task_manifest.jsonl, execution/model_configuration.json, and execution/execution_log.jsonl) are archived to permit third-party verification of accounting and reproducibility.

Analysis executor and inferential controls. The primary confirmatory analysis followed the preregistered paired_binary_analysis_v1 executor: compute contingency counts (n11,n10,n01,n00), absolute paired difference (guarded - baseline), perform an exact two-sided McNemar test for paired binary outcomes, and construct a paired percentile bootstrap 95% CI (seed = 1729, B = 10000). Secondary preregistered sensitivity analyses (failure-as-zero, stratum-wise paired tests with Benjamini–Hochberg FDR correction) were run as exploratory diagnostics; confirmatory claims rely only on the primary analysis. The analysis pipeline recorded and

archived its outputs (analysis/paired_contingency_table.csv, analysis/condition_summary.csv, analysis/missingness_summary.csv) and a reproducible analysis log (analysis/analysis_log.jsonl).

Missingness, exclusions, and reproducibility artifacts. The preregistered missingness plan required logging of provider/API technical failures and explicit treatment rules; no pairs were excluded under those rules in the confirmatory run. All raw responses, validator traces, repair traces, provider call traces, task manifests, and analysis artifacts referenced above are archived with machine-readable hashes. The archived initial_master_prompt reference path and SHA-256 (provenance/master_prompt.txt, SHA-256 = 1872df1e1805d2d9 6940456ca016bd665d1d5196add77f5acdf1582bb39b15ba) are included in the run bundle and the Disclosure Statement. The preregistration described an optional pilot (n = 60) for discordance estimation; that pilot was not executed prior to the main run (this deviation is recorded in the preregistration and Disclosure).

IV. Results

Execution integrity and accounting. The preregistered adapter completed the confirmatory run with completed_episode_count = 600, yielding complete_pair_count = 300 paired observations under the shared_initial protocol. Provider/audit records show hosted_model_call_event_count = 301 (300 shared_initial generations plus one repair invocation), cache_miss_count = 301, and stage_counts indicating shared_initial_generation = 300 and repair = 1. Deterministic reconciliation confirmed consistent episode accounting (all_deterministic_consistency_checks_passed = true) and the absence of provider/infrastructure failures or excluded pairs.

Primary confirmatory outcomes. The binary scoring produced baseline_success_count = 299 and guarded_success_count = 300 (analysis/condition_summary.csv), i.e., baseline = 299/300 (0.9966666667) and guarded = 300/300 (1.0). The paired contingency table (analysis/paired_contingency_table.csv) is n11 = 299, n10 = 1, n01 = 0, n00 = 0. The preregistered exact two-sided McNemar test (operating on the discordant counts n10 and n01) returned p = 1.0. The absolute paired improvement (guarded - baseline) equals 0.0033333333333332993. The paired bootstrap percentile 95% confidence interval computed with seed = 1729 and B = 10000 is [0.00, 0.01]. Because discordance is minimal (m = n10 + n01 = 1), the exact test has limited sensitivity and the bootstrap CI bounds the plausible absolute improvement to at most one percentage point in this experimental context.

Per-stratum diagnostics. Per the task manifest stratification, archived per-stratum tallies were: low difficulty baseline = 100/100 and guarded = 100/100; medium baseline = 100/100 and guarded = 100/100; high baseline = 99/100 and guarded = 100/100. Thus all discordant evidence is concentrated in the high stratum (1 rescues), while low and medium strata were at ceiling under both conditions. These counts are reported in the archived condition_summary.csv and corroborate that the observed discordance derives from a single hard example under the synthetic generator's difficulty taxonomy.

Repair diagnostic and episode-level trace. The solitary discordant pair invoked the permitted automated repair. Validator traces (deterministic_netops_validator_v5, validator_version = 5.0) attribute the initial failure to a sequencing invariant violation (the preregistered must_be_down_for_fields rule): the shared_initial candidate omitted an explicit 'admin down' step prior to a VLAN/MTU change required by the intent. The constrained repair operator performed a single localized insertion of the missing sequencing command, after which the deterministic validator reported valid == true for the corrected candidate. The repair therefore produced a deterministic rescue with one additional model/repair stage call. The repair provider trace and the validator's before/after traces for that episode are archived; the manuscript conveys the semantic summary rather than reproducing verbatim provider logs in the body.

Contamination, missingness, and negative controls. The preregistered contamination checks produced an empty contamination_summary.csv (no contamination flags). The missingness summary reports zero failed or unscorable episodes and zero excluded pairs under preregistered rules (analysis/missingness_summary.csv). Negative controls included by the preregistration did not cause unexpected pass behavior in the main run; any negative-control anomalies would have been treated as contamination per the preregistration and are not present in the archived run.

Cost and operational footprint. The guarded pipeline's deterministic rescue required a single additional repair stage call across 300 pairs (model_calls_used = 301), indicating that bounded automated repair under the chosen edit operator imposes minimal extra model-call cost in this run. The empirical repair_stage_count = 1 (analysis artifact) quantifies the low marginal runtime/model cost for deterministic rescue under the chosen repair budget and edit operator design.

Statistical interpretation and limitations of inference. The confirmatory inferential outcome (p = 1.0) primarily reflects the paucity of discordant pairs under the shared_initial estimand and chosen synthetic suite. Exact McNemar inference depends only on n10 and n01; with n10 = 1 and n01 = 0 the two-sided exact p equals 1.0, indicating no statistically detectable difference under conventional two-sided testing in this sample. The paired bootstrap CI (seed = 1729, B = 10000) provides a complementary bound for effect size: the absolute paired improvement is plausibly between 0 and 0.01 given observed data and the preregistered estimand. These inferential statements apply strictly to the shared_initial conditioning (one LLM draw reused across conditions), the limited repair budget (<=1 constrained edit), the synthetic task distribution, and the declared generator (gpt-5-mini) and validator (deterministic_netops_validator_v5).

Practical diagnostic takeaway. Under operational policies that restrict model calls per change request, the guarded generate→validate→repair pattern produced a deterministic rescue in one hard example with negligible additional

model-call expenditure in this synthetic suite. However, because baseline single-shot generation was near ceiling (299/300), the marginal observable benefit of bounded deterministic repair was correspondingly small in absolute terms. The archived per-episode validator traces and repair transcript enable forensic inspection to inform whether richer repair operators, iterative repair budgets, or alternative estimand conditioning (e.g., allowing resampling) would be expected to materially increase rescue rates in different deployment regimes.

V. Discussion

We expand the discussion with technical diagnostics, artifact-grounded interpretation, and operational guidance that follow directly from the archived execution and analysis artifacts.

1) Concrete diagnostics from archived artifacts. Audit fields in the execution manifest quantify cost and intervention footprint: model_calls_used = 301 (300 shared_initial + 1 repair), completed_episode_count = 600, and repair stage_count = 1. The paired_contingency_table.csv records n11 = 299, n10 = 1, n01 = 0, n00 = 0 and the per-stratum summary shows baseline/guarded counts of (100/100, 100/100, 99/100 vs. 100/100 across low/medium/high). The deterministic validator (deterministic_netops_validator_v5) applied the preregistered full_intent_constraint_validation checks (sequencing invariants, required_command_count, preserve_unspecified_state, and topology reachability invariants encoded in the synthetic suite). The single rescued failure was a sequencing omission (must_be_down_for_fields) corrected by the constrained repair operator; the repair consumed the single extra model call recorded. These concrete, archived diagnostics show exactly how often the bounded repair operator was invoked and what it corrected in the run.

2) Estimand conditioning and what the results mean operationally. The shared_initial estimand conditions on a single LLM draw per intent and therefore measures only deterministic verifier/repair rescue applied to that same candidate. Practically, this corresponds to operational policies that limit model calls for cost, latency, or governance reasons. Under such policies the guarded pipeline can only correct defects that are detectable and fixable by a bounded, localized edit applied to the same candidate. In our run, that policy permitted one deterministic rescue (sequencing insertion) at very low extra model cost (one additional call). By contrast, operational pipelines that allow resampling, verifier-guided regeneration, or interactive steering change the estimand: they can exploit generator variance and typically achieve higher pass rates, but at the cost of additional model calls and different governance tradeoffs. Evaluations must therefore align the estimand with the intended deployment policy; the archived artifacts provide a reproducible bound for the single-call policy case.

3) Ceiling effects, pilot utility, and experiment design lessons. The preregistration targeted a detectable paired difference of 10 percentage points with power $\approx$0.8 conditioned on pilot discordance estimates. The optional pilot (n = 60) described in the preregistration was not executed; as archived, the main run encountered a near-ceiling baseline (299/300) producing only one discordant observation and correspondingly low sensitivity to small absolute differences. This concrete outcome underscores a methodological lesson: when generator single-shot performance is uncertain, a small calibration pilot is efficient and often necessary to estimate discordance rates and select an appropriate confirmatory N. The archived design and the omitted pilot are documented in preregistration artifacts and should guide follow-ups: run a small pilot to estimate discordance, then freeze confirmatory N accordingly to avoid underpowered confirmatory runs in ceiling conditions.

4) Validator expressiveness and failure-mode dependence. Deterministic_netops_validator_v5 enacted a specific set of checks; on our synthetic suite the dominant observed failure class was sequencing omissions. Other potential failure classes—topology reachability violations, ACL misconfigurations, or forbidden-template edits—were not observed at scale here. This demonstrates that a guarded pipeline's marginal utility depends directly on the joint distribution of generator error types and validator coverage: if failure mass concentrates in classes the validator detects and the repair operator can fix, bounded deterministic rescue is effective; if failures are in classes the validator cannot detect (or repair operator cannot safely edit), the guarded pipeline provides mostly detection rather than automated rescue. The archived contamination_summary.csv (empty) and the negative controls embedded in the task manifests provide concrete, testable evidence that the synthetic suite exercised a limited set of failure modes for this run.

5) Repair budget tradeoffs and operational cost accounting. The run's provider audit shows that each automated repair attempt corresponds to an extra hosted model call (repair_stage_count = 1 consumed one call). Therefore, scaling the repair budget or permitting iterative repair increases model calls (and thus cost/latency) approximately linearly per allowed repair round under the same repair implementation. Organizations must trade off budgeted model calls per change against expected rescue yield conditioned on anticipated failure modes. The archived provider auditing fields (hosted_model_call_event_count, cache_miss_count) make this tradeoff explicit and reproducible for cost modeling.

6) Reproducibility, artifact utility, and forensic analysis. The run archives the full task manifests, shared_initial responses, validator traces, and the single repair provider trace for the rescued episode. These artifacts enable forensic inspection of exact edits, the validator state transitions (validation_before/validation_after), and the deterministic reconciliation records (all_deterministic_consistency_checks_passed = true). For practitioners, the availability of these low-level artifacts supports independent re-scoring, alternative validator checks, and ablation analyses (e.g., applying a different repair operator or richer validator to the same shared_initial candidates).

7) Threats to validity and how they map to artifact ev-

idence. Internal validity: provider nondeterminism was low (hosted_model_call_event_count = 301, failed calls = 0); execution accounting artifacts show no technical exclusions. Construct validity: our binary unflagged/pass scoring depends on validator coverage as instantiated by deterministic_netops_validator_v5; the synthetic task generator encoded domain invariants used by the validator, which tightens construct alignment but limits generality to other validator definitions. External validity: results are limited to gpt-5-mini and the supplied synthetic_topology_suite; generalizing to different LLMs, richer validators, or real operational networks requires follow-up runs using the same preregistration discipline. Conclusion validity: the near-ceiling baseline produced sparse discordance, constraining inferential power for small effects; the paired bootstrap CI and exact McNemar p documented in analysis artifacts quantify that limitation.

8) Practical recommendations grounded in archived evidence. (a) Align experimental estimands with deployment policies: if budgets constrain calls, evaluate shared_initial rescue; if resampling is allowed, design a different preregistration capturing resampling gains. (b) Run a small pilot to estimate discordance and calibrate confirmatory N; the preregistered but unexecuted pilot in our artifacts illustrates the cost of skipping calibration. (c) Make validator coverage explicit and test it with negative controls targeting diverse failure classes; archived negative controls and contamination artifacts support this practice. (d) Treat repair budget as a tunable parameter with explicit cost accounting: use provider audit fields to model marginal cost per repair call.

Taken together, these artifact-grounded diagnostics and recommendations show how a reproducible pairing of generator, verifier, and constrained repair can be quantitatively audited and how evaluators should select estimands, pilots, and validator tests to obtain operationally meaningful conclusions.

VI. Limitations

- Synthetic benchmark: tasks and topologies were generated by the preregistered deterministic task generator; real-world heterogeneity may expose different failure modes and increase verifier marginal utility.
- Single model and validator: only gpt-5-mini and deterministic_netops_validator_v5 were evaluated; results may not generalize to other LLMs or richer/diverse validators.
- Ceiling effect: baseline single-shot performance was near 100% on this suite (299/300), producing limited discordance and reduced power to detect small absolute improvements relative to larger pre-specified targets.
- Pilot not executed: the preregistered pilot intended for discordance estimation and power calibration was not run; no post-preregistration power recalculation was performed.
- Estimand conditioning: the shared_initial protocol conditions the estimand on a single LLM candidate and therefore does not measure verifier benefit that would arise from allowing independent alternative LLM candidates per condition (resampling or iterative generation).
- Repair budget: the guarded condition permitted at most one automated repair per pair; larger or iterative repair strategies may yield different outcomes.

VII. Conclusion

We executed a preregistered, artifact-archived paired comparison between a verifier-assisted pipeline (deterministic validator + <=1 automated repair) and accepting a single-shot LLM candidate under the shared_initial estimand on 300 synthetic NetOps intents using gpt-5-mini and deterministic_netops_validator_v5. Baseline single-shot generation produced 299/300 unflagged verifier passes and the guarded pipeline 300/300, yielding an absolute paired improvement of 0.00333 (exact McNemar two-sided p = 1.0; 95% bootstrap CI [0.00,0.01]). Deterministic validator+repair rescued a single sequencing omission under the bounded repair budget. The near-ceiling baseline limits inferential sensitivity to small absolute improvements under the shared_initial estimand. We release full preregistration, execution, and analysis artifacts to support reproducibility and targeted follow-up studies that vary estimand conditioning, model strength, task distribution, or repair budget.

Disclosure Statement

This experiment and manuscript were produced by an autonomous CNSM Agentic-AI research run executed under the archived initial master prompt (provenance/master_prompt.txt, SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human role: the human Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved pre-lock infrastructure; all literature search after the autonomy lock, autonomous study selection, preregistration (study id cnsmp2026-llm-verifier-predeployment-001), execution, analysis, manuscript drafting, AI peer review, and final compilation were performed autonomously under the locked master prompt. Models and orchestration: the declared generation model used was gpt-5-mini (execution/model_configuration.json declares requested_model = "gpt-5-mini"); adapter/orchestration framework: hosted_netops_gvr_v1. Validator and tooling: deterministic_netops_validator_v5 performed semantic and syntactic validation according to the preregistered full_intent_constraint_validation rule; the guarded condition permitted at most one automated repair call per pair implemented as a constrained edit operator. Preregistration: the confirmatory design, estimand, analysis executor, exclusion/missingness rules, and multiplicity plan were frozen prior to the main run and archived with the study id (preregistration SHA-256 = 425cb1652354120cf48f686b964723d64c76d0a8abe53cd5b5ad0eaa1af71fc7). Execution summary (archived): planned_episode_count = 600, completed_episode_count = 600, complete_pair_count = 300, model_calls_used = 301; provider audit: hosted_model_call_event_count = 301, stage_counts show shared_initial_generation = 300 and repair = 1. Pilot: the

preregistration described an optional pilot (n = 60) for discordance estimation and power calibration; that pilot was not executed and no post-preregistration recalculation of required sample size was performed. Deviations: no post-preregistration changes to the scientific design were made; no pairs were excluded. Human editing: no human manually edited or rewrote technical manuscript content after the autonomy lock. Artifact availability: all raw outputs, logs, task manifests, validator traces, repair provider traces, and analysis artifacts referenced in this manuscript are archived in the run bundle associated with the study id. The archived run bundle contains the low-level repair provider trace documenting the single automated repair and subsequent validation pass; this manuscript reports a concise semantic summary of that repair in Results rather than reproducing full verbatim provider transcripts in the body.

REFERENCES

[1] Yajie Zhou, Kevin Hsieh, Sathiya Kumaran Mani, Srikanth Kandula, Zaoxing Liu, "MeshAgent: Enabling Reliable Network Management with Large Language Models", 2025, 10.1145/3771567

[2] Xiaolong Wei, Yuehu Dong, Xingliang Wang, Xingyu Zhang, Zhejun Zhao, Dongdong Shen, Long Xia, Dawei Yin, "Beyond ReAct: A Planner-Centric Framework for Complex Tool-Augmented LLM Reasoning", 2026, 10.1609/aaai.v40i40.40676

[3] Xu Liu, Peng Zhang, Anubhavnidhi Abhashkumar, Jiawei Chen, Weirong Jiang, "Automatic Configuration Repair", 2024, 10.1145/3696348.3696895

[4] Hao Zhou, Chengming Hu, Ye Yuan, Yufei Cui, Yili Jin, Can Chen, Haolun Wu, Dun Yuan, Li Jiang, Di Wu, Xue Liu, Jianzhong Zhang, Xianbin Wang, Jiangchuan Liu, "Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities", 2024, 10.1109/comst.2024.3465447

[5] X X Zhang, Xianming Gao, Peilin Tao, Tao Feng, "GraphSynth: Synthesis of Network Configuration Templates Using Large Language Models", 2025, 10.1145/3728725.3728742

[6] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, Yarin Gal, "Detecting hallucinations in large language models using semantic entropy", 2024, 10.1038/s41586-024-07421-0

[7] Yunze Wei, Wei, Kaiwen, Shaohui Du, Wang, Jianyu, Liu, Zhangzhong, Yawen Wang, Zhenxin Li, Congcong Miao, Xiaohui Xie, Yong Cui, "Automated Network Protocol Testing with LLM Agents", 2025, 10.48550/arxiv.2510.13248

[8] Fuliang Li, Haozhi Lang, Jiajie Zhang, Jiaxing Shen, Xingwei Wang, "PreConfig: A Pretrained Model for Automating Network Configuration", 2024, 10.48550/arxiv.2403.09369